

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Radioelektroniki i Techniki Multimedialnych

Praca końcowa

Studia Podyplomowe

Głębokie Sieci Neuronowe - Zastosowania w Mediach Cyfrowych

Grafowe sieci neuronowe w przewidywaniu zdarzeń klinicznych

Marta, Monika

Numer albumu XXXX

promotor

YYYYY

WARSZAWA 2026

Grafowe sieci neuronowe w przewidywaniu zdarzeń klinicznych

Streszczenie.

Głównym celem pracy jest Nacisk w pracy został położony na ... W pierwszej części pracy przedstawiono ... Następnie opisano ... Druga część pracy rozpoczyna się od ... Eksperymenty opisane w dalszej części pracy przedstawiają ... Dodatkowym etapem pracy jest ... W zakończeniu pracy przedstawiono podsumowanie wykonanych działań oraz dalsze działania, które mogą przynieść poprawę ...

Słowa kluczowe: NFT, ChatGPT, Buzzword300

Graph neural networks in predicting clinical events

Abstract.

Summary in English.....

Keywords: ChatGPT, Buzzword300



.....
miejscowość i data

.....
imię i nazwisko studenta

.....
numer albumu

.....
kierunek studiów

OŚWIADCZENIE

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Jednocześnie oświadczam, że:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płyce kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

.....
czytelny podpis studenta

Spis treści

1. Wstęp	9
Wykaz symboli i skrótów	10
2. Main objective	11
3. Przegląd literatury	12
3.1. Architecture	13
3.1.1. RGCN	13
3.1.2. Transformer	13
3.2. HCR	14
3.2.1. Hidden link reconstuction	14
3.2.2. Endpoint prediction	14
3.2.3. Oversmoothing methods	15
4. Dataset	16
4.1. Synthethic dataset	16
4.1.1. Data generation	16
4.1.2. Knowledge graph description	16
4.1.3. Patient data description	16
4.1.4. Data Characteristics	16
4.2. Proxy dataset	16
4.2.1. Last-FM dataset	16
4.2.2. Hetionet knowledge graph	16
5. Main Framework	17
5.1. Link prediction in Graph Reconstruction	18
5.2. Endpoint prediction	18
5.2.1. Training Dataset - Data Representation	18
5.2.2. Model architecture	18
5.2.3. HCR application	21
5.2.4. Training setup	21
6. Experiments results	23
6.1. Link prediction in graph reconstuction	23
6.2. Endpoint prediction	23
6.2.1. Architecture	23
6.2.2. HCR configuration	26
6.2.3. Oversmoothing	26
7. Models benchmark	28
7.1. Link prediction	29
7.2. Endpoint prediction	30
7.2.1. Data Representation	30

7.2.2. Results	30
8. Conclusions	32
Spis rysunków	33
Spis tabel	33

1. Wstęp

Krótki opis dziedziny problemu.

Motywacja, dlaczego warto zająć się rozwiązaniem postawionego problemu.

Opis wkładu własnego - co zrobiono w ramach przygotowywania pracy dyplomowej.

Np. przygotowanie zbioru danych, opracowanie i implementacja architektury modelu itp.

YOLO **redmon2016you**

Wykaz symboli i skrótów

GNN – Graph Neural Network

DAG – Directed Acyclic Graph

HCR – Hierarchical Correlation Reconstruction

ADR – Adverse Drug Reaction

AKI – Acute Kidney Injury

DILI – Drug-Induced Liver Injury

GAT – Graph Attention Network

SAGE – Graph SAGE

RGCN – Relational Graph Convolutional Network

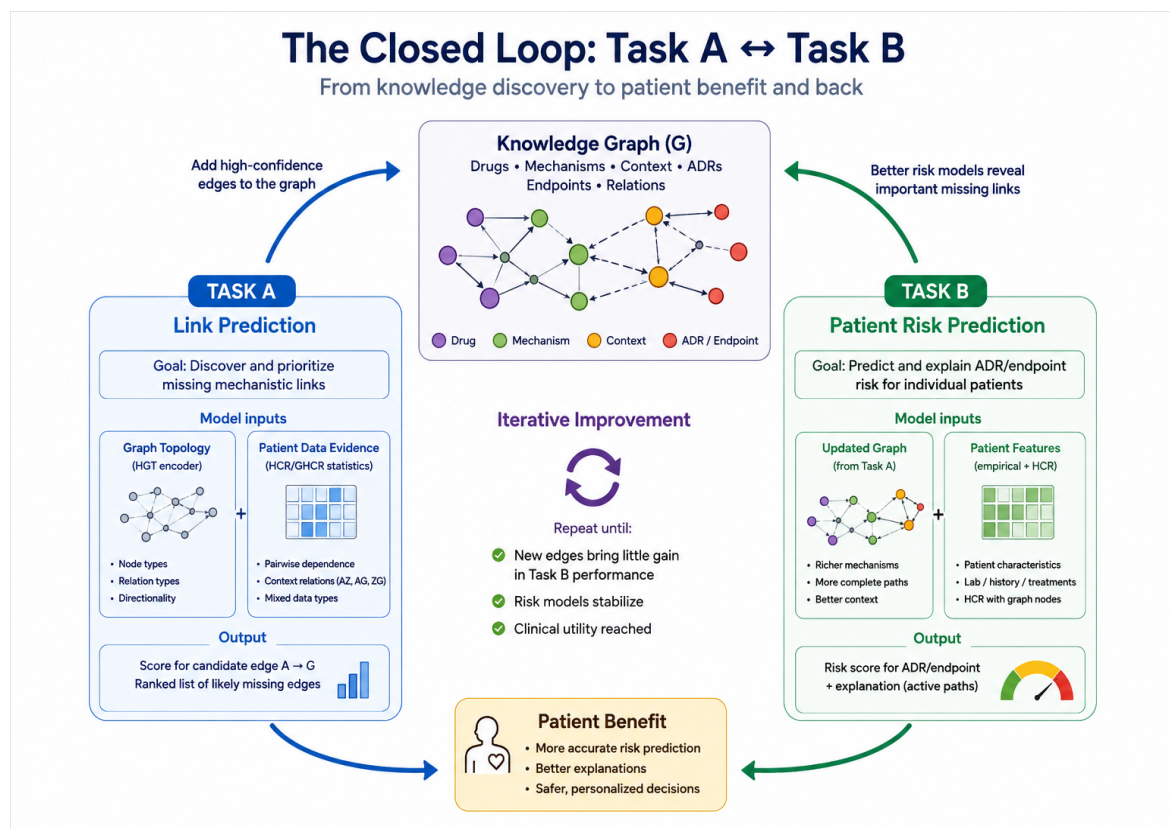
AUC – Area Under the Curve

AUPRC – Area Under the Precision-Recall Curve

2. Main objective

The aim of this thesis is to develop a graph neural network models that could be applied in two tasks: a) to predict the presence of hidden links in a causal graph, b) to predict the values of endpoints for entities based on tabular data represented as a causal graph.

The two tasks are complementary ... [we will add here how the two tasks relate to each other in medicine + picture that shows it] The models will be trained and evaluated on synthetic datasets, as well as on real-world datasets. The performance of the models will be compared with baseline methods, and the results will be analyzed to understand the strengths and limitations of the proposed approach.

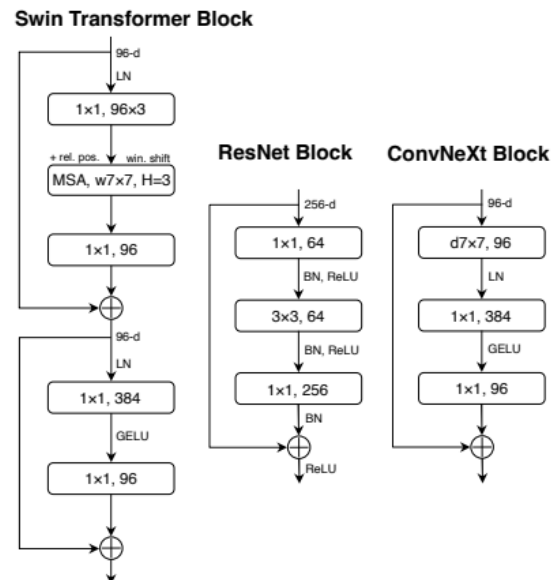


Rysunek 2.1. The description of the two tasks addressed in the thesis.

3. Przegląd literatury

Przegląd literatury związanej z rozwiązywanym problemem.

W pracy **he2016deep** opisano W **liu2022convnet** zaproponowano nowatorską architekturę.... Przykładowy rysunek 3.1.



Rysunek 3.1. Porównanie bloków rezydualnych wykorzystywanych w sieci Swin Transformer, ResNet i ConvNeXt.

3.1. Architecture

3.1.1. RGCN

3.1.2. Transformer

3.2. HCR

3.2.1. Hidden link reconstruction

3.2.2. Endpoint prediction

Endpoint prediction is a node classification tasks in data.

3.2.3. Oversmoothing methods

Residual connections switch the effective per-layer propagation operator from the (nilpotent, on a DAG without self-loops) adjacency operator, which is the standard formalization of the over-smoothing regime and is the mechanism through which multi-layer signal survives on a directed acyclic structure.

PairNorm (Zhao et Akoglu, 2020) re-centers and re-scales node representations after each layer to keep the pairwise distance between representations from collapsing, without preventing the underlying mixing.

DropEdge (Rong et al., 2020) randomly removes a fraction of edges independently in every layer during training, slowing the rate of information mixing and acting as a structural regularizer.

Jumping Knowledge (Xu et al., 2018) aggregates the outputs of all layers (via concatenation, cat, or elementwise max) rather than only the last one, letting a node fall back on a shallower, less-smoothed representation when the deepest layer has become uninformative.

4. Dataset

4.1. Synthethic dataset

The dataset includes two structures - knowledge graph of medicine and patient dataset with pateint's characteristics.

4.1.1. Data generation

4.1.2. Knowledge graph description

Tabela 4.1. Node types in the patient causal DAG.

Node type	Description
patient_context	Stable patient-level covariates (e.g. demographics, co-morbidities) available at the moment of prediction.
drug_exposure	Drugs the patient is exposed to; source nodes for pharmacotherapeutic causal paths.
mechanism	Intermediate physiological mechanisms mediating the effect of a drug or context on downstream states.
adr_or_intermediate_state	Intermediate adverse or physiological states positioned between mechanisms and clinical endpoints.
observation_or_selection	Nodes representing observation, documentation, or selection processes rather than a clinical state itself.
clinical_endpoint	Target adverse clinical outcomes (e.g. AKI, DILI, Hospitalization) predicted by the node classification head.

4.1.3. Patient data description

4.1.4. Data Characteristics

4.2. Proxy dataset

4.2.1. Last-FM dataset

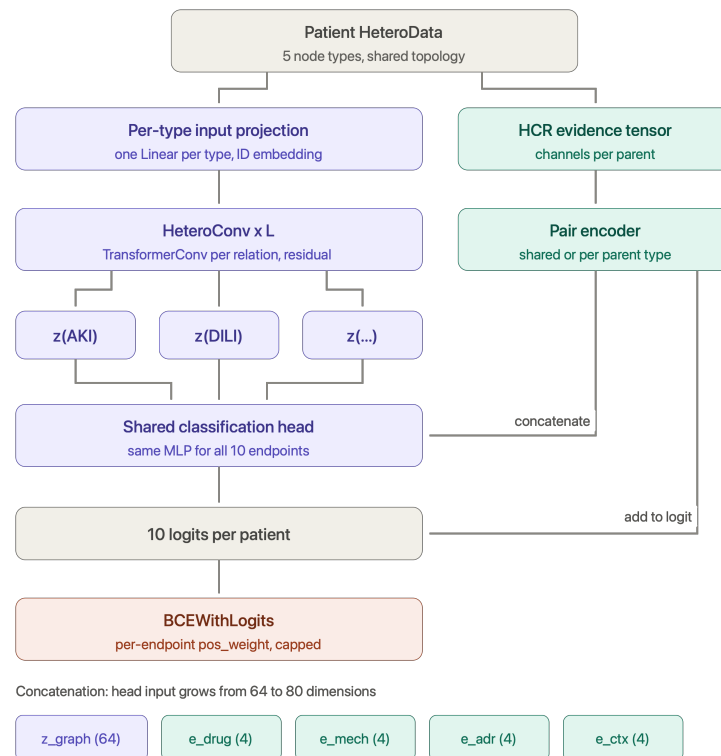
4.2.2. Hetionet knowledge graph

5. Main Framework

A causal knowledge graph is not merely a static record of the literature: it can be systematically audited, weighted, and enriched with observational data, and subsequently used as the basis for explainable clinical prediction of drug-related outcomes. This thesis operationalizes that idea through two complementary graph neural network tasks, applied to a shared causal knowledge graph of pharmacotherapy:

- Task A:** link prediction, which reconstructs and updates the knowledge graph based on real patient data;
- Task B:** node classification, which predicts patient-specific clinical endpoints based on the knowledge graph and real patient data.

Since both tasks operate on the same underlying knowledge graph, we adopt graph neural networks as the common modeling framework, and enrich them with Hierarchical Correlation Reconstruction (HCR). In fig. 5.1 there is presented high-level approach to construct GNN with HCR. While the two tasks share this common foundation, their implementation details differ in several respects; we therefore present each task separately in the following subsections.



Rysunek 5.1. The presentation of approach to node classification task.

5.1. Link prediction in Graph Reconstruction

5.2. Endpoint prediction

Task B is formulated as multi-label node classification on a heterogeneous, per-patient graph. Rather than building one global graph and treating patients as additional nodes, every patient is represented as a copy of the same fixed causal DAG topology, with node feature vectors instantiated from that patient's individual covariates. The prediction targets are a fixed subset of clinical events on patient site. Formally, for a patient graph $G_p = (V, E)$ shared across the cohort, with patient-specific node features x_p , the model computes

$$\hat{y}_{p,e} = \sigma(f_\theta(G_p, x_p)_e), \quad e \in \mathcal{E}_{\text{target}}$$

where f_θ is a heterogeneous graph neural network shared across all patients, and only the leaf-level target embeddings are read out by the classification head.

5.2.1. Training Dataset - Data Representation

The first thing in applying GNNs is to build properly the input into neural network. The input graph for the node classification task was built based on:

- a) **nodes**
- b) **edges**
- c) **per-scenario patient sample table**

Based on those items HeteroData object was created which was represented by objects type presented in the table

The patient data is encoded on nodes level. In fig. 5.1 there is presented the exemplary DAG for one of patient.

Two structural exclusion mechanisms are applied before any feature is computed:

- a) nodes flagged `exclude_from_observed_model` or `is_latent` in the audit (hidden confounders);
- b) nodes that are structural descendants of any endpoint in the DAG (e.g. documentation nodes such as `hospital_contact`, `fall_reported`, which are caused by the endpoint rather than causal parents of it)

are detected by graph descendant traversal and excluded from the feature set, even though they are not explicitly flagged in the source data. All normalization statistics (mean/std for continuous node values) and all HCR evidence statistics (section 5.2.3) are computed exclusively on the training split of patients, to prevent both node-level and patient-level information leakage across splits.

5.2.2. Model architecture

The core architecture is a stack of heterogeneous graph convolutional layers wrapped in HeteroConv, shared across relation types but parameterized identically for every pa-

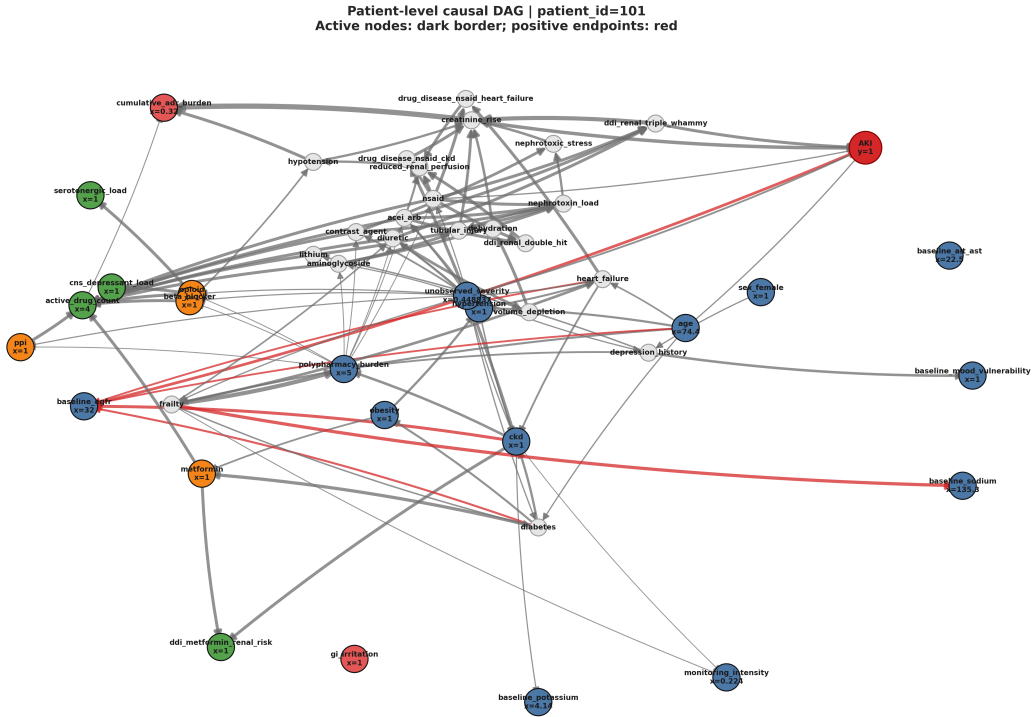
Tabela 5.1. Components of the per-patient HeteroData graph representation.

Component	Description
Node types	Six semantic categories of nodes shared across all patient graphs: <code>patient_context</code> , <code>drug_exposure</code> , <code>mechanism</code> , <code>adr_or_intermediate_state</code> , <code>observation_or_selection</code> , and <code>clinical_endpoint</code> .
Relations	Keyed by (<code>src_type</code> , "to", <code>dst_type</code>) weighted by <code>edge_attr</code> instead of using <code>edge_type</code> . This keeps the number of distinct convolutional relation modules in HeteroConv small while still letting edge-type information reach the message function.
Edge attributes (<code>edge_attr</code>)	A $1 + 3 + \text{edge types} $ -dimensional vector per relation: signed effect size, activation frequency, mechanistic confidence, evidence weight, and the edge-type one-hot. Convolutions consuming a scalar <code>edge_weight</code> (GraphConv/GCN) receive only the signed effect-size column; convolutions consuming a full <code>edge_attr</code> (GAT, Transformer) receive the whole vector.
Node features (<code>x</code>)	Static audited attributes plus a topological-layer encoding (numeric or onehot, configurable), combined with the patient's own observed value for that node where the node is visible under the current observability regime.

tient graph. Four convolution backbones are supported through a common registry and were benchmarked against each other: GraphSAGE, GraphConv (GCN-style), GATv2, and TransformerConv. Because these operators were originally designed for homogeneous graphs, each is instantiated once per edge type inside HeteroConv, so the heterogeneous DAG is handled without collapsing node/edge types into a single homogeneous graph. As benefit of casual graph structure is possibility to encode deep structured path for node classification, multiple number of layers was tested to choose the best model.

A design issue that was handled during experiments is the duplication of the self-transform across relations. Several PyG convolution layers (SAGEConv, TransformerConv) internally add their own linear self-loop term, wrapped per-relation inside HeteroConv, this term is silently duplicated once per incoming relation type for a given node type. The implementation removes this internal duplication (`root_weight=False` for SAGE/Transformer) and replaces it with a single, explicit, controlled self-transform per layer per node type, applied once in the shared encoder rather than once per relation. For GAT-family layers, where the self-dependency is baked into the attention query and cannot be disabled, the explicit self-transform acts as an additional controlled term on top of the irreducible built-in dependency. GraphConv/GCN, which has no

5. Main Framework



Rysunek 5.2. The description of the two tasks addressed in the thesis.

root_weight parameter at all, is deliberately excluded from this mechanism, since its built-in self-transform cannot be removed without reimplementing the layer.

A node-identity embedding, added after the input projection, was introduced to resolve a specific representational collapse. Message-passing GNNs are bounded in expressive power by the 1-dimensional Weisfeiler-Lehman graph isomorphism test: two nodes with identical initial features and structurally equivalent neighborhoods cannot be assigned distinct representations by any number of convolutional layers. In the dataset, several endpoint nodes of the same type (e.g. Serotonin syndrome, Rhabdomyolysis, Lactic acidosis) share identical static audited attributes (e.g. severity_weight, observability) and comparable topological positions, and were therefore indistinguishable to the model except through such positional cues. To break this symmetry, a learnable embedding table is maintained per node type, indexed by each node's fixed local index, and added to the projected input features before the first convolutional layer, following the same principle underlying position aware node embeddings in .

This connects directly to the model's handling of oversmoothing. Using multiple message-passing layers risks oversmoothing the embedding signal, where repeated aggregation causes node representations to converge toward indistinguishability. To mitigate this, several oversmoothing countermeasures were applied: residual connections, Jumping Knowledge, DropEdge, and PairNorm. These methods were evaluated both individually and in combination with the HCR enrichment described in section 5.2.3.

5.2.3. HCR application

In addition to the purely structural GNN pathway, approach presented in the thesis incorporates a statistical evidence side-channel - HCR described in details in section 3.2. This channel is motivated by the observation that some parent-endpoint dependencies are well captured by simple, well-understood pairwise statistics computed directly from the training cohort, and that combining such statistics with the deep GNN representation captures the strong signal from direct parents and smoothed signal from further layers. There were run couple of experiments with HCR configurations and GNNs. However, in general the approach was about computation of contingency-table statistics for each target endpoint and direct parents or pre-direct parents in the audited DAG (restricted to nodes with inferred binary, count, or continuous value types). The statistics computed was as followed per (parent, endpoint) pair: the ϕ -coefficient, normalized mutual information, a tanh-scaled log odds ratio, risk difference, and a tanh-scaled log lift, plus prevalence and support terms. There are nine to forty-one channels depending on configuration.

As these coefficients are computed with the patient's own endpoint label as part of the statistic, a naive computation would leak the label into the feature. This was addressed with an exact leave-one-out correction: every training patient's coefficient is computed from all other training patients only, using a closed-form re-weighting of the population statistic.

Continuous and count-valued parents are mapped to comparable pseudo-uniform scores via a train-only empirical CDF transform (with deterministic, patient-ID-seeded jitter for count variables), and Legendre polynomial bases are used to capture non-linear dependence on continuous parents.

Three integration modes were supported:

- a) linear approach - a single scalar HCR score added directly to the GNN logit,
- b) nonlinear encoder - parent-endpoint evidence vectors pooled and passed through a shared MLP
- c) separate encoders per parent node type - every direct parent passed through a separated MLP
- d) triple relations - relations not only focused on direct parents, but direct parents and grandparents.

The pooling in HCRPairEncoder masks out padding positions (endpoints with fewer parents than the batch's maximum) and returns a neutral zero vector for endpoints with no eligible parents, avoiding division by zero.

5.2.4. Training setup

Training iterates over patient graphs batched via PyTorch Geometric's DataLoader. The experiment were run with targeted binary cross-entropy and focal loss with class-balance-aware weighting. Weight and focal alpha were computed from the training

split's endpoint prevalence, since adverse events are rare and highly imbalanced. Model selection uses ReduceLROnPlateau on validation performance, and evaluation is reported separately per endpoint and in aggregate, using AUROC, AUPRC, and prevalence-normalized lift, alongside the representation-level over-smoothing diagnostics (MAD, cosine similarity, Dirichlet energy) used to relate architectural choices. Due to rare events the main metric for model choice was AUPRC.

6. Experiments results

6.1. Link prediction in graph reconstruction

6.2. Endpoint prediction

6.2.1. Architecture

The first set of experiments established how much the choice of message-passing operator matters for this task. Three convolutions were compared: R-GCN, GraphSAGE and TransformerConv. All three are defined for homogeneous graphs, so each was wrapped in HeteroConv, which instantiates a separate convolution for every relation type and aggregates the results at the target node and allows to preserve the heterogenous structure.

The experiments were conducted with the same GNN structure differentiated couple different hyperparameters. All details are presented in table 6.1.

Tabela 6.1. Hyperparameters shared by all architectures. Values were fixed across the experiments reported in this chapter

Group	Hyperparameter	Value
Encoder	Hidden units per convolutional layer	64
	Number of layers L	1, 2, 3, 4, 5, 6
	Aggregation across relations	sum
	Activation	ReLU
	Batch normalisation	yes
Regularisation	Dropout	0.2
	Weight decay	5×10^{-4}
Classification head	Number of layers	2
	Hidden units	64 (equal to encoder width)
	Output units	1 per endpoint
Optimisation	Optimiser	Adam
	Learning rate	1×10^{-3}
	Batch size	128
	Epochs (maximum)	200
	Loss	weighted BCE with logits
	Positive weight cap	30
	Gradient clipping	disabled
	Seed	42
Model selection	Selection metric	validation AUPRC
	Early-stopping patience	20 epochs
	Minimum improvement	0.001

All three convolutions are wrapped in HeteroConv, which instantiates a separate convolution for each of the 19 relation types in the graph. The parameter counts therefore

6. Experiments results

scale with the number of relations. In attention-based layers the hidden width is split across heads, so the output width stays at 64 regardless of the number of heads.

Tabela 6.2. Architecture-specific hyperparameters.

Hyperparameter	R-GCN	GraphSAGE	TransformerConv
Attention heads	—	—	4
Units per head	—	—	16
Output width per relation	64	64	$4 \times 16 = 64$
Basis decomposition	8 bases	—	—
Uses edge attributes	indirectly, via relation id	no	yes, via <code>edge_dim</code>
Weight matrices per relation	shared through bases	2	3 (query, key, value)

The results for those experiments are reported in table 6.3. The following conventions apply to this and all subsequent result tables. *Lift* is the AUPRC divided by the prevalence baseline, that is, the factor by which the model improves over a random ranking; at a mean prevalence of 5.5% a lift of 2.3 corresponds to an AUPRC of 0.125. *% ceiling* is defined as

$$\frac{\text{AUC} - 0.5}{\text{AUC}_{\text{Bayes}} - 0.5} \cdot 100\%, \quad (1)$$

where $\text{AUC}_{\text{Bayes}} = 0.651$ is the Bayes ceiling averaged over the same eight endpoints. It expresses the share of attainable signal recovered by the model. Both terms are subtracted by 0.5 so that a random classifier scores 0%. Unless stated otherwise, the epoch is selected on the validation set and test metrics are reported from that epoch. In table 6.3 there are included only layers 1,3 and 6 to show the trend with the increased number of layers.

Without residual connections the three architectures diverge sharply at $L=6$, collapse to constant predictions ($\text{AUC}=0.50$).

That’s why the next step was application of residual connections described in section 3.2.3 to see the capabilities of models with higher number of layers. The results are presented in table 6.4.

With residual connections all three architectures converge to comparable performance (table 6.4): 69–80% of the Bayes ceiling on the test set, with differences of about 0.01 AUC. The more informative factors for model assessment are AUPRC and Lift, which show that the model performs nearly 2.5 times better than a random baseline. Since the architectures achieve comparable performance, the choice for the remaining experiments was therefore made on structural grounds. TransformerConv is the only one of the three that conditions message passing on edge attributes, which the audited DAG provides (mechanistic confidence, evidence weight, effect size, and sign). Since the role of edge-level

Tabela 6.3. Message-passing architectures without residual connections.

Architecture	L	Validation				Test			
		AUC	AUPRC	Lift	% ceil.	AUC	AUPRC	Lift	% ceil.
R-GCN	1	0.623	0.124	2.28	81.6	0.624	0.120	2.21	82.0
GraphSAGE	1	0.622	0.124	2.28	80.9	0.619	0.120	2.21	78.9
TransformerConv	1	0.622	0.125	2.30	81.2	0.620	0.120	2.20	79.6
R-GCN	3	0.542	0.076	1.40	28.0	0.542	0.068	1.26	27.7
GraphSAGE	3	0.549	0.079	1.45	32.2	0.549	0.072	1.32	32.8
TransformerConv	3	0.550	0.083	1.53	33.2	0.552	0.073	1.35	34.5
R-GCN	6	0.503	0.055	1.00	2.2	0.500	0.054	1.00	0.0
GraphSAGE	6	0.530	0.065	1.20	20.0	0.510	0.061	1.12	6.6
TransformerConv	6	0.527	0.065	1.20	18.0	0.500	0.054	1.00	0.0

Tabela 6.4. The same architectures with residual connections.

Architecture	L	Validation				Test			
		AUC	AUPRC	Lift	% ceil.	AUC	AUPRC	Lift	% ceil.
R-GCN	1	0.625	0.125	2.30	82.7	0.625	0.121	2.22	83.2
GraphSAGE	1	0.621	0.124	2.27	80.6	0.619	0.120	2.21	78.7
TransformerConv	1	0.623	0.126	2.31	81.3	0.619	0.120	2.21	79.0
R-GCN	2	0.628	0.133	2.43	85.2	0.628	0.124	2.28	85.3
R-GCN	3	0.630	0.133	2.44	86.0	0.627	0.125	2.29	84.2
GraphSAGE	3	0.623	0.132	2.42	81.4	0.616	0.124	2.27	77.0
TransformerConv	3	0.628	0.135	2.47	85.2	0.620	0.123	2.27	79.7
R-GCN	5	0.631	0.133	2.44	86.9	0.630	0.124	2.27	86.3
R-GCN	6	0.629	0.135	2.48	85.6	0.626	0.123	2.27	83.9
GraphSAGE	6	0.628	0.134	2.46	84.9	0.624	0.127	2.33	82.3
TransformerConv	6	0.627	0.134	2.46	84.2	0.620	0.122	2.24	79.4

evidence is important for causal classification, TransformerConv was selected for all subsequent experiments with HCR.

6.2.2. HCR configuration

HCR was introduced to supply the model with statistical evidence about parent–endpoint dependence that the message-passing path has to infer on its own (section 5.2.3). The configurations examined here vary along three axes, each addressing a separate question. The first is the form of the wide branch: a single learned coefficient per endpoint applied to a precomputed dependence score, against a small nonlinear encoder applied to each parent separately. The second is the scope of the evidence: binary parents only, or all node types, in which case continuous and count variables are expanded in a Legendre basis. The third is the order of the statistic: pairwise coefficients between a parent and an endpoint, or third-order moments over grandparent–parent–endpoint paths, which capture interactions that pairwise coefficients cannot represent. All tested configurations are listed in table 6.5.

Without HCR the model degrades sharply with depth and predicts a constant at $L = 6$. Every HCR variant recovers a substantial part of the lost performance, and the gain grows with depth similarly like residual connections.

Tabela 6.5. HCR variants on TransformerConv *without* residual connections. Without HCR the model degrades sharply with depth and predicts a constant at $L = 6$; every HCR variant recovers a substantial part of the lost performance, and the gain grows with depth.

L	HCR	Validation				Test			
		AUC	AUPRC	Lift	%ceil.	AUC	AUPRC	Lift	%ceil.
1	—	0.622	0.1251	2.30	81.19	0.620	0.1197	2.20	79.61
1	linear 42D	0.622	0.1247	2.29	81.10	0.617	0.1191	2.19	77.83
1	typed+triple	0.630	0.1331	2.44	86.30	0.623	0.1229	2.26	81.40
2	typed+triple	0.628	0.1314	2.41	84.67	0.621	0.1228	2.26	80.13
3	—	0.550	0.0833	1.53	33.19	0.552	0.0734	1.35	34.49
3	linear 42D	0.604	0.1173	2.15	69.28	0.612	0.1031	1.90	74.51
3	triple	0.602	0.1131	2.08	67.92	0.590	0.1021	1.88	59.81
3	typed+triple	0.618	0.1265	2.32	78.41	0.616	0.1214	2.23	77.02
4	typed+triple	0.630	0.1369	2.51	86.09	0.616	0.1239	2.28	76.97
5	typed+triple	0.627	0.1308	2.40	84.25	0.617	0.1222	2.25	77.61
6	—	0.527	0.0653	1.20	17.96	0.500	0.0544	1.00	0.00
6	linear 42D	0.616	0.1206	2.21	76.91	0.603	0.0955	1.76	68.22
6	triple	0.607	0.1154	2.12	70.71	0.579	0.0967	1.78	52.56
6	typed+triple	0.622	0.1222	2.24	80.91	0.612	0.1179	2.17	74.53

Once the residual path restores the flow of information through depth, no HCR variant improves on the model without it. All configurations fall within 70–80% of the Bayes ceiling on the test set. It shows that HCR is a type of enrichment that works similarly like residual connections.

6.2.3. Oversmoothing

As collapse was observed

Tabela 6.6. HCR variants on TransformerConv *with* residual connections. Once the residual path restores the flow of information through depth, no HCR variant improves on the model without it; all configurations fall within 70–80% of the Bayes ceiling on the test set.

L	HCR	Validation				Test			
		AUC	AUPRC	Lift	%ceil.	AUC	AUPRC	Lift	%ceil.
1	—	0.623	0.1255	2.31	81.30	0.619	0.1202	2.21	79.02
1	nonlin. bin.	0.622	0.1238	2.27	81.12	0.617	0.1176	2.16	77.82
1	nonlin. 42D	0.621	0.1250	2.30	80.56	0.616	0.1186	2.18	76.95
1	typed+triple	0.623	0.1238	2.27	81.43	0.617	0.1181	2.17	77.60
3	—	0.628	0.1346	2.47	85.23	0.620	0.1233	2.27	79.66
3	nonlin. bin.	0.627	0.1339	2.46	84.06	0.617	0.1242	2.28	77.48
3	nonlin. 42D	0.630	0.1343	2.47	85.95	0.619	0.1247	2.29	78.80
3	typed+triple	0.622	0.1325	2.43	80.94	0.617	0.1237	2.27	77.48
6	—	0.627	0.1337	2.46	84.19	0.620	0.1219	2.24	79.37
6	nonlin. bin.	0.614	0.1177	2.16	75.90	0.605	0.1082	1.99	69.84
6	nonlin. 42D	0.624	0.1301	2.39	82.21	0.618	0.1214	2.23	78.23
6	typed+triple	0.626	0.1299	2.39	83.73	0.618	0.1222	2.25	78.08

Tabela 6.7. Oversmoothing metrics

L	Residual	Metoda	MAD	CosSim	Rank	DirE	Std	E_L/E_0	MAD_L/MAD_0
1	no	—	0.882	0.571	0.229	0.178	0.168	0.115	1.585
3	no	—	0.799	0.630	0.125	0.041	0.083	0.057	0.898
3	no	JK cat	0.837	0.603	0.070	0.098	0.088	0.018	1.138
3	no	JK max	0.766	0.669	0.208	0.271	0.197	0.019	1.088
3	no	DropEdge	0.750	0.652	0.135	0.053	0.086	0.042	0.871
3	no	JK cat + DropEdge	0.827	0.610	0.073	0.073	0.082	0.020	1.093
3	no	JK max + DropEdge	0.727	0.687	0.190	0.175	0.153	0.024	1.161
3	yes	—	0.774	0.689	0.180	0.189	0.182	0.098	1.112
3	yes	JK cat	0.826	0.630	0.104	0.219	0.174	0.038	1.024
3	yes	JK max	0.630	0.816	0.315	0.337	0.219	0.059	1.039
3	yes	DropEdge	0.784	0.682	0.169	0.110	0.143	0.102	1.102
3	yes	JK cat + DropEdge	0.826	0.633	0.099	0.215	0.177	0.047	0.984
3	yes	JK max + DropEdge	0.678	0.799	0.271	0.279	0.206	0.066	1.094
6	no	—	0.808	0.634	0.125	0.049	0.094	0.080	0.781
6	no	JK cat	0.813	0.611	0.037	0.053	0.056	0.012	1.068
6	yes	—	0.756	0.702	0.133	0.096	0.135	0.056	1.055
6	yes	JK cat	0.823	0.652	0.056	0.163	0.127	0.017	0.939

7. Models benchmark

results for proxy dataset

7.1. Link prediction

7.2. Endpoint prediction

7.2.1. Data Representation

7.2.2. Results

Tabela 7.1. Results under full observability on the proxy dataset. The epoch was selected on the validation set. -- in the HCR column denotes a model without the HCR branch. Lift is AUPRC divided by the prevalence baseline. Jumping knowledge was not used in any configuration.

<i>L</i>	Residual	HCR	Validation			Test		
			AUC	AUPRC	Lift	AUC	AUPRC	Lift
1	no	—	0.757	0.188	7.92	0.664	0.120	6.49
1	no	nonlin. 9D	0.763	0.207	8.75	0.657	0.120	6.47
1	no	typed 41D	0.759	0.194	8.17	0.663	0.116	6.29
1	yes	—	0.756	0.196	8.26	0.669	0.106	5.72
1	yes	nonlin. 9D	0.756	0.201	8.49	0.670	0.135	7.31
1	yes	typed 41D	0.754	0.185	7.79	0.671	0.124	6.71
3	no	—	0.756	0.199	8.38	0.722	0.152	8.24
3	no	nonlin. 9D	0.805	0.275	11.62	0.686	0.169	9.14
3	no	typed 41D	0.812	0.243	10.26	0.678	0.154	8.32
3	yes	—	0.799	0.225	9.48	0.685	0.166	8.97
3	yes	nonlin. 9D	0.814	0.235	9.90	0.714	0.183	9.92
3	yes	typed 41D	0.802	0.255	10.76	0.678	0.151	8.16
6	no	—	0.610	0.121	5.11	0.585	0.077	4.14
6	no	nonlin. 9D	0.782	0.210	8.86	0.680	0.174	9.42
6	no	typed 41D	0.794	0.235	9.89	0.680	0.126	6.84
6	yes	—	0.817	0.255	10.77	0.728	0.173	9.36
6	yes	nonlin. 9D	0.832	0.310	13.06	0.760	0.179	9.70
6	yes	typed 41D	0.815	0.284	11.99	0.744	0.239	12.91

Tabela 7.2. Results under restricted (*bedside*) observability, $L = 3$.

Residual	HCR	Validation		Test		
		AUC	Lift	AUC	AUPRC	Lift
no	—	0.568	4.20	0.556	0.062	3.36
no	lin. 41D	0.730	10.09	0.681	0.150	8.10
no	nonlin. 41D	0.751	9.39	0.678	0.154	8.35
no	typed 41D	0.753	9.45	0.679	0.135	7.31
yes	—	0.568	4.39	0.555	0.076	4.14
yes	lin. 41D	0.732	10.78	0.683	0.168	9.10
yes	nonlin. 41D	0.754	10.24	0.679	0.164	8.89
yes	typed 41D	0.754	10.18	0.678	0.173	9.33

Tabela 7.3. Representation smoothing metrics under full observability, measured on the validation set. By the conventional reading, lower MAD and higher cosine similarity indicate stronger smoothing. Validation AUC is repeated for comparison. Note that residual configurations appear more smoothed by every metric while performing better.

L	Residual	HCR	MAD	Cos. sim.	Cos. sim. (ep.)	Dirichlet	AUC
1	no	—	0.7691	0.6216	0.8246	0.2706	0.757
1	no	nonlin. 9D	0.7629	0.6142	0.7844	0.2495	0.763
1	no	typed 41D	0.7684	0.6191	0.8148	0.2706	0.759
1	yes	—	0.7363	0.7042	0.7606	0.5724	0.756
1	yes	nonlin. 9D	0.7448	0.6946	0.7060	0.5048	0.756
1	yes	typed 41D	0.7410	0.7026	0.7899	0.5022	0.754
3	no	—	0.6515	0.6798	0.9582	0.1175	0.756
3	no	nonlin. 9D	0.7067	0.6641	0.8365	0.0904	0.805
3	no	typed 41D	0.7235	0.6314	0.7333	0.0972	0.812
3	yes	—	0.4842	0.8319	0.7772	0.3712	0.799
3	yes	nonlin. 9D	0.5517	0.7782	0.7752	0.3905	0.814
3	yes	typed 41D	0.5113	0.8275	0.7132	0.4044	0.802
6	no	—	0.7060	0.6530	0.8604	0.1024	0.610
6	no	nonlin. 9D	0.7101	0.6495	0.7815	0.0892	0.782
6	no	typed 41D	0.7222	0.6408	0.8657	0.0949	0.794
6	yes	—	0.5634	0.7975	0.7949	0.3143	0.817
6	yes	nonlin. 9D	0.5030	0.8174	0.6658	0.2636	0.832
6	yes	typed 41D	0.5623	0.8095	0.8472	0.3930	0.815

8. Conclusions

Podsumowanie i krytyczna analiza osiągniętych rezultatów. Ocena, czy osiągnięto założony cel pracy. Dyskusja co można było zrobić lepiej i propozycja dalszych prac w celu usprawniania opracowanego rozwiązania.

Spis rysunków

2.1	The description of the two tasks addressed in the thesis.	11
3.1	Porównanie bloków rezydualnych wykorzystywanych w sieci Swin Transformer, ResNet i ConvNeXt.	12
5.1	The presentation of approach to node classification task.	17
5.2	The description of the two tasks addressed in the thesis.	20

Spis tabel

4.1	Node types in the patient causal DAG.	16
5.1	Components of the per-patient HeteroData graph representation.	19
6.1	Hyperparameters shared by all architectures. Values were fixed across the experiments reported in this chapter	23
6.2	Architecture-specific hyperparameters.	24
6.3	Message-passing architectures without residual connections.	25
6.4	The same architectures with residual connections.	25
6.5	HCR variants on TransformerConv <i>without</i> residual connections. Without HCR the model degrades sharply with depth and predicts a constant at $L = 6$; every HCR variant recovers a substantial part of the lost performance, and the gain grows with depth.	26
6.6	HCR variants on TransformerConv <i>with</i> residual connections. Once the residual path restores the flow of information through depth, no HCR variant improves on the model without it; all configurations fall within 70–80% of the Bayes ceiling on the test set.	27
6.7	Oversmoothing metrics	27
7.1	Results under full observability on the proxy dataset. The epoch was selected on the validation set. -- in the HCR column denotes a model without the HCR branch. Lift is AUPRC divided by the prevalence baseline. Jumping knowledge was not used in any configuration.	30
7.2	Results under restricted (<i>bedside</i>) observability, $L = 3$	31
7.3	Representation smoothing metrics under full observability, measured on the validation set. By the conventional reading, lower MAD and higher cosine similarity indicate stronger smoothing. Validation AUC is repeated for comparison. Note that residual configurations appear more smoothed by every metric while performing better.	31